📁 Python File & Path Management — Quick Professional Notes

Goal: Understand how Python locates, creates, reads, writes, and manages files/folders — from beginner usage to professional project structure.

AI Product Developer Connection: Used in subtitle processing, user uploads, JSON storage, logs, generated AI outputs, configuration, caching, and backend systems.

---

1. 🧠 Core Concept — File System

Python works with two main things:

```
File System
│
├── 📁 Directory / Folder
│       └── Contains files & folders
│
└── 📄 File
        ├── .json
        ├── .srt
        ├── .txt
        ├── .eps
        └── etc.
```

Path

A **path** tells Python *where something is located*.

```
Relative Path → data/vocabulary.json
Absolute Path → C:\Projects\AIApp\data\vocabulary.json
```

---

2. 🛠️ `pathlib` — Modern Professional Approach

`pathlib` is Python's modern, object-oriented module for working with files and directories.

```
from pathlib import Path
```

Core syntax

Code	Meaning
`Path("data")`	Represents a path
`Path("data/file.json")`	Represents a file path
`path / "file.json"`	Builds a child path
`Path.cwd()`	Current working directory
`Path(__file__)`	Current Python source file
`Path(__file__).parent`	Folder containing source file
`path.resolve()`	Absolute path
`path.exists()`	Check existence
`path.is_file()`	Check whether it's a file
`path.is_dir()`	Check whether it's a directory
`path.mkdir()`	Create directory

Example

```
from pathlib import Path

BASE = Path(__file__).parent
DATA = BASE / "data"

DATA.mkdir(exist_ok=True)

VOCAB = DATA / "vocabulary.json"

print(VOCAB.resolve())
```

Important

```
Path("data")
```

❌ does **not** create `data`.

```
Path("data").mkdir()
```

✅ creates `data`.

---

3. 📍 Understanding Locations

Current Working Directory

```
Path.cwd()
```

Means:

Where was this program launched from?

```
Terminal
   ↓
CWD
   ↓
Relative paths are interpreted from here
```

Python File Location

```
Path(__file__).parent
```

Means:

Where does this `.py` file live?

Critical distinction

```
Path.cwd()
     ↓
Where I RUN from

__file__.parent
     ↓
Where Python FILE lives
```

They **can be different**.

---

4. 🛣️ Relative vs Absolute Paths

Type	Example	Meaning
Relative	`"data/file.json"`	Relative to current location
Absolute	`C:\Projects\App\data\file.json`	Complete location

Professional approach

Avoid hardcoding:

```
"C:\\Users\\Khushi\\Desktop\\..."
```

Prefer:

```
BASE = Path(__file__).parent
file = BASE / "data" / "vocabulary.json"
```

This makes your application more portable.

---

5. 📁 Directory Creation

Create one folder

```
Path("data").mkdir()
```

Don't fail if it already exists

```
Path("data").mkdir(exist_ok=True)
```

Nested directories

```
Path("data/vocabulary").mkdir(parents=True, exist_ok=True)
data/
└── vocabulary/
```

Meaning

```
mkdir()
   ↓
Make Directory

exist_ok=True
   ↓
Already exists? → Don't raise error

parents=True
   ↓
Create missing parent folders too
```

---

6. 📄 File Creation & File Modes

Python's basic file operation:

```
open(path, mode)
```

Modes

Mode	Meaning	If missing	Existing data
`"r"`	Read	❌ Error	Preserved
`"w"`	Write	✅ Creates	⚠️ Overwrites
`"a"`	Append	✅ Creates	Preserved + adds
`"x"`	Exclusive create	✅ Creates	❌ Error

Example

```
with open(file_path, "w") as file:
    file.write("Hello")
```

`with` automatically handles closing the file.

---

7. 🔄 Why Deleted Files Come Back

If the program contains:

```
with open("data.json", "w") as file:
    ...
```

then every execution performs:

```
Run program
    ↓
open()
    ↓
File exists?
   / \
 NO   YES
 ↓     ↓
Create  Clear old content
   \    /
    ↓
 Write new data
```

Therefore:

The file doesn't recreate itself. The program recreates it.

This applies to:

```
.json
.txt
.srt
.eps
.csv
.log
```

etc.

---

8. 🗂️ JSON Persistence

JSON is commonly used for lightweight application data.

```
import json
```

Python → JSON

```
with open("data.json", "w") as file:
```

```
    json.dump(data, file, indent=4)
Dictionary
    ↓
json.dump()
    ↓
JSON file
```

JSON → Python

```
with open("data.json", "r") as file:
    data = json.load(file)
JSON file
    ↓
json.load()
    ↓
Dictionary
```

Professional rule

Don't blindly do:

```
open("user_data.json", "w")
```

for important user data.

First consider:

```
if not file_path.exists():
```

because `"w"` can overwrite existing data.

---

9. 🐍 Dictionary → Persistent Data

```
data = {
    "name": "Khushi"
}

data["level"] = "Python Beginner"
```

Add

```
data["level"] = "Beginner"
```

Update

```
data["name"] = "New Name"
Key doesn't exist → ADD
Key exists        → UPDATE
```

Then:

```
json.dump(data, file, indent=4)
```

saves it permanently.

---

10. 🖥️ `os` Module vs `pathlib`

`os` is Python's older/general operating-system interface.

```
import os
```

Common `os` operations

`os`	Purpose
`os.getcwd()`	Current working directory
`os.makedirs()`	Create directories
`os.path.exists()`	Check existence
`os.path.join()`	Combine paths
`os.path.dirname()`	Get parent directory

Equivalent modern approach

```
# os
os.path.join(folder, "data.json")

# pathlib
folder / "data.json"
```

Recommendation

```
Beginner:
os + open()
```

```
Professional modern Python:
pathlib + open()
```

You should **understand both**, but prefer `pathlib` for new Python projects.

---

11. 📐 Professional Project Structure

For your **AI Subtitle Learning Platform**:

```
AI_Subtitle_App/
│
├── app.py
│
├── input/
│   └── episode01.srt           ← User input
│
├── data/
│   ├── vocabulary.json         ← Persistent user data
│   └── learning_history.json   ← Persistent user data
│
├── output/
│   └── translated_episode01.srt ← Generated output
│
├── logs/
│   └── application.log
│
└── cache/
    └── temporary_data.json
```

Data ownership matters

Category	Example	Treatment
User input	`episode01.srt`	Preserve
User data	`vocabulary.json`	Never blindly overwrite
Generated output	translated `.srt`	Usually regeneratable
Logs	`.log`	Append/rotate
Cache	temporary data	Safe to recreate

File extension alone does NOT determine whether a file is temporary.

---

12. 🔥 Professional File-Safety Pattern

```
from pathlib import Path
import json

BASE = Path(__file__).parent
DATA = BASE / "data"
DATA.mkdir(exist_ok=True)

VOCAB = DATA / "vocabulary.json"

if not VOCAB.exists():
    data = {
        "name": "Khushi",
        "level": "Python Beginner"
    }

    with open(VOCAB, "w") as file:
        json.dump(data, file, indent=4)
```

Workflow

```
Application starts
        ↓
Locate project directory
        ↓
Create data/ if missing
        ↓
Check vocabulary.json
        ↓
Missing? ──YES──→ Create
        │
        NO
        ↓
Preserve existing data
```

---

13. ⚠️ Common Mistakes

❌ Mixing strings and Path

```
"data" / "file.json"
```

✅

```
Path("data") / "file.json"
```

❌ Assuming `Path()` creates folders

```
Path("data")
```

✅

```
Path("data").mkdir(exist_ok=True)
```

❌ Blindly overwriting user data

```
open("vocabulary.json", "w")
```

Better

```
if not file.exists():
    ...
```

❌ Assuming CWD = Python file directory

```
cwd ≠ __file__.parent
```

They **may** be the same, but aren't guaranteed to be.

---

14. 🔗 5-Minute Command Flow

```
WHERE AM I?
     ↓
Path.cwd()
     ↓
WHERE IS MY PYTHON FILE?
     ↓
Path(__file__).parent
     ↓
```

```
CREATE PATH
      ↓
Path("data") / "file.json"
      ↓
CREATE FOLDER
      ↓
mkdir(exist_ok=True)
      ↓
CHECK
      ↓
exists() / is_file() / is_dir()
      ↓
ABSOLUTE LOCATION
      ↓
resolve()
      ↓
FILE OPERATION
      ↓
open(path, mode)
      ↓
  ┌─────┬─────┬─────┬─────┐
  r     w     a     x
  ↓     ↓     ↓     ↓
read write append create
      ↓
JSON?
      ↓
json.dump() ← Python → JSON
json.load() ← JSON → Python
```



Interview Flash Revision

Q: `Path()` VS `mkdir()`?
`Path()` represents a location; `mkdir()` creates the directory.

Q: `cwd()` VS `__file__.parent`?
CWD is where the program is launched from; `__file__.parent` is where the Python file lives.

Q: Why does a deleted JSON file return?
Because the program contains code that recreates/writes it when executed.

Q: Why is `"w"` dangerous for user data?
It truncates existing content before writing.

Q: `json.dump()` vs `json.load()`?
`dump()` saves Python data to JSON; `load()` reads JSON into Python.

Q: `os` or `pathlib`?
Know both; prefer `pathlib` for modern Python path management.

🧠 Final Memory Formula

```
PATH
↓
WHERE?

mkdir()
↓
FOLDER

open()
↓
FILE

r / w / a / x
↓
HOW?

json.dump()
↓
SAVE

json.load()
↓
RESTORE

exists()
↓
SAFE?

cwd()
↓
WHERE RUN?

__file__.parent
↓
```

WHERE CODE LIVES?

AI Product Developer connection

```
File System
     ↓
Path Management
     ↓
Input / Output Organization
     ↓
Persistent JSON
     ↓
User Data Safety
     ↓
Uploads / Logs / Cache
     ↓
Database Concepts
     ↓
Backend
     ↓
AI Product
```